

Static Methods

A **static method** in Java is a method that belongs to the class rather than to any specific object. This means that static methods can be called without creating an instance of the class. Static methods are defined using the `static` keyword in Java.

Characteristics of Static Methods:

1. **Belongs to the Class:** Static methods belong to the class and are shared across all instances of the class.
2. **Can be Called Without an Object:** You do not need to create an object of the class to call a static method. You can call it directly using the class name.
3. **Cannot Access Instance Variables:** Static methods can only access other static variables and static methods. They cannot access instance variables or instance methods directly.
4. **Used for Utility or Helper Methods:** Static methods are often used for utility functions or methods that don't need to modify or access object-specific data.

Syntax for Declaring a Static Method:

```
public class ClassName {  
    public static returnType methodName(parameters) {  
        // method body  
    }  
}
```

- **returnType:** The return type of the method (e.g., `int`, `String`, `void`, etc.).
- **methodName:** The name of the method.
- **parameters:** The parameters passed to the method (optional).

Example of Static Method in Java

Here's an example that demonstrates the use of static methods in Java:

```
public class StaticMethodExample {  
  
    // Static method to calculate the square of a number  
    public static int square(int num) {  
        return num * num;  
    }  
}
```

```

// Static method to calculate the cube of a number
public static int cube(int num) {
    return num * num * num;
}

public static void main(String[] args) {
    // Calling static methods directly using the class name without
    creating an object

    int number = 5;

    // Calling the static method 'square' and printing the result
    int squareResult = StaticMethodExample.square(number);
    System.out.println("The square of " + number + " is: " +
squareResult);

    // Calling the static method 'cube' and printing the result
    int cubeResult = StaticMethodExample.cube(number);
    System.out.println("The cube of " + number + " is: " + cubeResult);
}
}

```

Explanation:

1. **Static Methods:** The methods `square(int num)` and `cube(int num)` are declared as `static`. This allows them to be called without creating an instance of the `StaticMethodExample` class.
2. **Calling Static Methods:** In the `main()` method, the static methods are called using the class name: `StaticMethodExample.square(number)` and `StaticMethodExample.cube(number)`. This is because static methods can be accessed through the class itself.
3. **Output:**

```

The square of 5 is: 25
The cube of 5 is: 125

```

The static methods perform calculations (square and cube) on the number and return the results.

Key Points to Remember:

1. **Accessing Static Methods:** Static methods can be accessed using the class name (recommended) or via an object (although it's not required and not recommended).
2. **No `this` Keyword:** Inside a static method, you cannot use the `this` keyword because

`this` refers to the current object, but static methods do not belong to objects.

3. **Static Methods Can Call Other Static Methods:** A static method can call other static methods in the same class directly.

Summary:

- Static methods are associated with the class itself rather than an instance of the class.
- They are useful for operations that do not require access to instance variables or methods.
- You can call static methods without creating an object of the class, using the class name itself.